CareMate — Project Guide

This document describes the current CareMate project: its purpose, technology, data flow, APIs, source files, setup, and known limitations. It reflects the implementation in this repository and is intended as a handoff and development reference.

1. What CareMate does

CareMate is a medication tracking and care coordination web application. The **Caretaker account represents the patient** and owns their patient profile. A **Caregiver account monitors** a patient after requesting access and receiving the patient's approval.

The application lets users maintain medication plans, review OCR-extracted prescription details, check scheduled doses, record doses as taken or missed, view adherence reports, configure follow-ups and reminders, and receive in-app notifications. The interface offers English, Hindi, and Marathi preferences; translation coverage is currently partial.

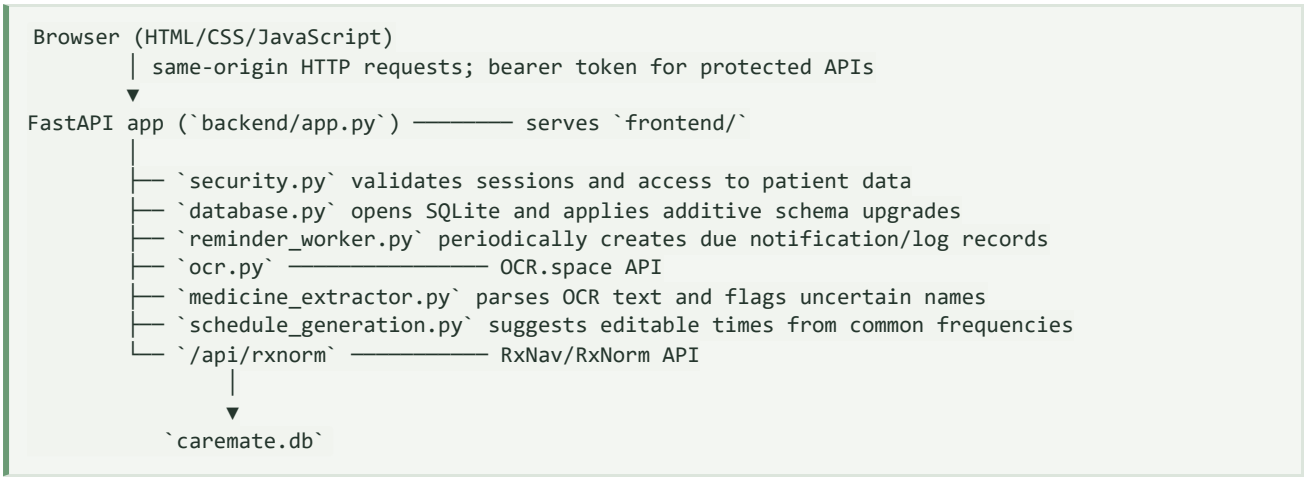
CareMate is a record-keeping and reminder aid. It does not prescribe medication, diagnose conditions, or replace a clinician or pharmacist.

2. Technology used

Area	Technology	How the project uses it
Backend/API	Python, FastAPI, Pydantic	HTTP routes, request validation, OpenAPI docs, and static web hosting
ASGI server	Uvicorn	Runs the FastAPI application locally
Database	SQLite (`caremate.db`)	Stores accounts, sessions, patient profiles, medicines, schedules/logs, reminders, and notifications
Frontend	HTML, CSS, vanilla JavaScript	Single-page interface served by the FastAPI app; uses `fetch` for API calls
Password security	Python `hashlib.scrypt` , `secrets` , `hmac`	Salted password hashes and opaque bearer sessions; only a token hash is stored in the database
Prescription OCR	OCR.space HTTP API	Reads text from an uploaded image; requires an `OCR_SPACE_API_KEY` environment variable
Medicine identity lookup	RxNorm/RxNav REST API	Optional name lookup through the application's `/api/rxnorm` endpoint
HTTP client	`requests`	Calls OCR.space and RxNav
Upload parsing	`python-multipart`	Receives prescription image uploads

There is no separate frontend framework, Node backend, hosted database, email/SMS provider, push-notification service, or LLM integration in the current implementation. The care assistant endpoint uses conservative built-in responses and urgent-phrase detection.

3. Architecture and request flow



The app serves the web interface and JSON API from the same origin. FastAPI's interactive API documentation is available at `/docs` while the server is running.

Typical prescription-to-reminder flow

1. The user uploads a prescription image to the patient's OCR route.
2. `ocr.py` sends the image to OCR.space using the configured key.
3. `medicine_extractor.py` parses possible medicine names, strengths, frequencies, duration, and timing instructions. It may offer a dictionary-based name correction when the optional `medicinedictionary.txt` file is present in the parent project directory.
4. The frontend shows extracted results for review. OCR does **not** create a medication plan automatically.
5. The user confirms and saves a medication plan. Suggested reminder times are defaults that the user must review.
6. The daily schedule endpoint combines active medication plans and dose logs. A user or caregiver with access records a dose.
7. The background worker writes due reminders and notifications to SQLite. The open browser polls notifications and can show desktop notifications if permission is granted.

4. Main source files

File	Responsibility
`app.py`	Small root launcher that imports `backend.app:app`, preserving the familiar `uvicorn app:app` command
`backend/app.py`	FastAPI application, request models, API routes, authorization integration, report calculations, frontend hosting, and reminder-worker lifespan
`backend/database.py`	SQLite connection helper, table creation, additive legacy-schema migrations, and database-level medicine/log helpers
`backend/security.py`	Password hashing, session creation/validation, HTTP bearer authentication, and patient-access checks
`backend/reminder_worker.py`	Background polling for medication reminders, missed-dose records, refill alerts, custom reminders, and appointment notifications

File	Responsibility
`backend/ocr.py`	OCR.space HTTP client; reads `OCR_SPACE_API_KEY` from the environment
`backend/medicine_extractor.py`	Conservative parsing of common prescription text and optional local dictionary matching
`backend/schedule_generation.py`	Suggests editable default reminder times for common frequency patterns such as `1-0-1`
`backend/rxnorm.py`	RxNorm helper used by the legacy CLI and kept import-safe; the web API has its own `/api/rxnorm` route in `backend/app.py`
`scripts/legacy/process_prescription.py`	Standalone older prescription-processing CLI flow
`scripts/legacy/adherence.py`, `report.py`, `reminder.py`	Earlier standalone utilities; current web flows are primarily implemented in `backend/app.py` and `backend/reminder_worker.py`
`scripts/legacy/update_time.py`	Standalone time/update utility from the prototype
`scripts/legacy/view_database.py`	Local database inspection helper; use only on a trusted machine because the database can contain personal data
`frontend/index.html`	Frontend document shell and stylesheet/script references
`frontend/app.js`	Browser state, rendering, API requests, forms, dose actions, notifications, and language strings
`frontend/app.css`	Responsive layout, components, and visual styling
`requirements.txt`	Python dependencies
`README.md`	Short setup and API overview
`prescription.jpg`	Local sample image used by the standalone OCR helper; keep only if it is still useful for manual development
`caremate.db`	Existing SQLite application data; **do not delete** as routine cleanup

Current folder layout

```
CareMate_ocr/
├── app.py                      # root Uvicorn compatibility launcher
├── backend/                     # FastAPI app and Python runtime modules
├── frontend/                    # static browser application
├── scripts/legacy/              # older standalone scripts, outside the web request path
├── caremate.db                  # existing SQLite data; retained at the project root
├── requirements.txt
├── README.md
└── CAREMATE_PROJECT_GUIDE.md
```

5. Accounts, authorization, and data model

Roles

- **Caretaker:** the patient account. Registration creates an associated patient profile. They manage their own care data and approve or decline caregiver requests.
- **Caregiver:** requests access using the email address of a Caretaker account. Access is granted only after approval. A caregiver can view and monitor connected patient records.

Protected API calls use `Authorization: Bearer <access_token>`. Sessions expire after 14 days. The backend verifies patient access for patient and medicine routes. Passwords are stored as salted scrypt hashes; session tokens are stored as hashes.

Principal SQLite tables

- `users`, `sessions`: user accounts and revocable sessions.
- `patients`, `patient_caregivers`, `caregiver_requests`: profile ownership, approved connections, and access requests.
- `prescriptions`: OCR text and prescription upload metadata.
- `medicines`, `medication_logs`: medication plans and scheduled dose outcomes.
- `appointments`, `reminders`: follow-up appointments and custom scheduled reminders.
- `notifications`: durable in-app notification records.
- `contact_messages`: messages submitted through the contact route.

On startup `database.py` creates missing tables and applies additive migrations for older prototype databases. It is designed to preserve existing medicine and dose history. Back up `caremate.db` before manual database changes.

6. API reference

All routes below are served under the same origin. Unless listed as public, API routes require a valid bearer token and appropriate patient access. Request/response schemas can be inspected at `/docs`.

Health, language, and authentication

Method	Route	Purpose
`GET`	`/health`	Simple service health response
`GET`	`/api/languages`	Lists supported language codes and names
`POST`	`/api/auth/register`	Creates account and session; Caretaker registration also creates a patient profile
`POST`	`/api/auth/login`	Authenticates and returns a bearer token
`GET`	`/api/auth/me`	Returns the authenticated user and profile information
`POST`	`/api/auth/logout`	Revokes the current session
`POST`	`/api/auth/change-password`	Changes password and invalidates previous sessions
`PATCH`	`/api/users/{user_id}`	Updates the authenticated user's editable profile/preferences

Supported language codes: `en` (English), `hi` (Hindi), `mr` (Marathi).

Patients and caregiver access

Method	Route	Purpose
`POST`	`/api/patients`	Creates a patient profile where permitted by role
`GET`	`/api/patients`	Lists patients visible to the current user

Method	Route	Purpose
`GET`, `PATCH`	`/api/patients/{patient_id}`	Reads or updates an accessible profile
`GET`	`/api/caregivers`	Lists caregiver-related information permitted by the API
`POST`	`/api/caregiver-requests`	Caregiver requests access using the patient's account email
`GET`	`/api/caregiver-requests`	Lists requests relevant to the current user
`PATCH`	`/api/caregiver-requests/{request_id}`	Patient accepts or declines a request
`PUT`, `DELETE`	`/api/patients/{patient_id}/caregivers/{caregiver_id}`	Manages a caregiver connection where allowed

Prescriptions and medicines

Method	Route	Purpose
`POST`	`/api/patients/{patient_id}/prescriptions/ocr`	Uploads an image as multipart field `image`; returns OCR and extracted candidates for review
`GET`	`/api/patients/{patient_id}/prescriptions`	Lists accessible prescription records
`GET`	`/api/prescriptions/{prescription_id}`	Reads one accessible prescription record
`POST`, `GET`	`/api/patients/{patient_id}/medicines`	Creates or lists medication plans
`PATCH`, `DELETE`	`/api/medicines/{medicine_id}`	Updates or pauses a medication plan
`GET`	`/api/patients/{patient_id}/schedule?on=YYYY-MM-DD`	Returns the selected day's scheduled doses and statuses
`POST`	`/api/medicines/{medicine_id}/doses`	Records a dose as `taken`, `missed`, or `skipped`
`GET`	`/api/patients/{patient_id}/doses`	Lists dose logs with optional date filters

Medication plans include name, dose, frequency, timing instructions, course duration, start date, reminder times (`HH:MM`), and optional quantity/RxNorm identifier.

Reports, appointments, reminders, and notifications

Method	Route	Purpose
`GET`	`/api/patients/{patient_id}/reports/adherence`	Returns adherence totals, missed-dose summary, and estimated supply/course progress
`POST`, `GET`	`/api/patients/{patient_id}/appointments`	Creates or lists follow-ups
`PATCH`, `DELETE`	`/api/appointments/{appointment_id}`	Updates or removes an appointment
`POST`, `GET`	`/api/patients/{patient_id}/reminders`	Creates or lists custom reminders

Method	Route	Purpose
`PATCH`, `DELETE`	`/api/reminders/{reminder_id}`	Updates or removes a custom reminder
`GET`	`/api/notifications`	Lists notifications visible to the current user; supports `unread_only` and patient filtering
`PATCH`	`/api/notifications/{notification_id}/read`	Marks a notification read

Support and integrations

Method	Route	Purpose
`POST`	`/api/contact`	Accepts a contact message (public route)
`POST`	`/api/chat`	Returns built-in conservative care guidance and urgent-phrase flag (public route)
`GET`	`/api/rxnorm?name=...`	Looks up a medicine name through the RxNav/RxNorm REST API

7. Reminders and delivery behavior

- FastAPI starts `reminder_worker.py` as a background task when its application starts.
- The worker checks due items every 20 seconds by default (`CAREMATE_REMINDER_POLL_SECONDS`). It records missed doses after a 30-minute grace period by default (`CAREMATE_DOSE_GRACE_MINUTES`). Both values can be configured with environment variables.
- The browser polls while the app is open and shows due medicine information in the interface. Browser desktop alerts require the user to grant notification permission.
- The API process and browser must be available for the corresponding behavior. There is no remote push, SMS, or email delivery when the app is closed.
- A dose marked taken is written to `medication_logs` with its scheduled date/time and taken timestamp. Reports use these logs.

8. Local setup and run

From PowerShell, in the `CareMate_ocr` folder:

```
 py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
$env:OCR_SPACE_API_KEY = "your-ocr.space-api-key"
uvicorn app:app --reload
```

Then open:

- App: `http://127.0.0.1:8000/`
- API docs: `http://127.0.0.1:8000/docs`
- Health: `http://127.0.0.1:8000/health`

The OCR key is optional for the rest of the application but required for prescription image OCR. Configure it in the process environment; do not commit a real key to source control or project documentation. SQLite defaults to `caremate.db` in this folder. Set `CAREMATE_DB` to use another database path. `CAREMATE_CORS_ORIGINS` is only

needed when serving the frontend from a separate origin.

9. Cleanup and generated files

Python `__pycache__/` folders contain disposable bytecode and are not source files. The local `.venv/` folder contains installed dependencies; it is not source code, but keeping it avoids reinstalling packages and is useful for running this checkout. Neither should be included in a source-control archive. The SQLite file, prescription sample, and standalone scripts are retained because they may contain user data or useful prototype functionality.

10. Known limitations and next work

- Interface translation is partial; some labels and content remain English after choosing Hindi or Marathi.
- OCR results and suggested medicine corrections can be wrong. Review against the prescription and seek qualified advice for unclear instructions.
- Schedule defaults only cover common frequency patterns and are suggestions, not dosing guidance.
- Refill quantity and remaining-course calculations are estimates based on entered data.
- Browser notifications are not a guaranteed alarm service; this is not a medical-grade monitoring system.
- The app is a local prototype. Before handling real patient data in production, add HTTPS deployment, backup/restore procedures, secret management, rate limiting, security review, monitoring, and a formal privacy/compliance assessment.
- No automated test suite is currently documented in this project.

11. Detailed code walkthrough

This section explains the execution path in the current source code, including what each important helper is responsible for and how a typical browser action travels through the application.

11.1 Application startup (`app.py`)

`backend/app.py` contains the FastAPI application. The small project-root `app.py` imports and re-exports that app so the existing command `uvicorn app:app` continues to work from the project root.

1. It imports package-relative helpers such as the SQLite module, OCR client, schedule generator, authentication middleware, and reminder worker.
2. It creates a FastAPI instance and registers CORS middleware for configurable development origins.
3. It registers the custom HTTP authentication middleware. The middleware is implemented in `security.py` and is responsible for enforcing authentication and patient access before protected API handlers run.
4. The FastAPI lifespan function starts `run_reminder_worker()` as an asynchronous background task. When the server shuts down, it cancels that task cleanly.
5. Near the end of `backend/app.py`, `StaticFiles` mounts the project-level `frontend/` directory at `/static` and mounts its HTML entry point at `/`. This is why a separate frontend server is not needed in local development.

Routes are ordinary Python functions decorated with FastAPI annotations such as `@app.get(...)` and `@app.post(...)`. Pydantic classes (`UserIn`, `MedicineIn`, `DoseIn`, etc.) validate JSON request bodies before handler code runs. The handlers return dictionaries/lists, which FastAPI serializes as JSON.

11.2 Database lifecycle (`database.py`)

`DB_PATH` points to the path from `CAREMATE_DB`, or defaults to `caremate.db` in the project root, one level above the `backend/` package. The `connection()` context manager:

- Opens a SQLite connection with a timeout and row objects that can be accessed by column name.
- Enables SQLite foreign-key enforcement for that connection.
- Commits if the code inside the context finishes successfully.
- Rolls back if an exception escapes the context.
- Closes the connection in all cases.

`create_tables()` runs at module import. Its `CREATE TABLE IF NOT EXISTS` statements create new installations. It then checks old schemas using `PRAGMA table_info(...)` and adds new columns with `ALTER TABLE` when needed. This is an additive migration strategy: it does not rebuild or clear the database. The legacy `medication_logs.medicine_name` field is retained for compatibility with older rows; current dose writes account for this older schema while saving the medicine ID as well.

The module also keeps `add_medicine()` and `record_status()` helpers for direct/older application flows. The main browser API generally performs its SQL in the relevant route handler so it can validate the patient and return the saved record.

11.3 Authentication and access checks (`security.py`)

Password creation: `hash_password()` rejects values outside the configured length range, generates a random salt, and uses `hashlib.scrypt`. The returned database value contains the algorithm label, salt, and derived digest; the original password is not stored.

Password verification: `verify_password()` parses the stored value, derives a digest from the submitted password and stored salt, and compares digests with `hmac.compare_digest()` to avoid a normal early-exit string comparison.

Session creation: `new_session()` creates a long random bearer token and stores only its SHA-256 hash alongside the user ID and expiration timestamp. The plain token is returned once to the browser. The browser stores it in local storage and sends it in the `Authorization` header.

Per-request authentication: `auth_middleware()` permits public routes such as registration, login, contact, and the built-in chat. Other `/api/` calls must have a valid bearer session. It resolves the token to a user and checks IDs in patient-scoped URLs, medicine URLs, and other resource URLs. `enforce_patient_access()` permits the patient account owner or a caregiver whose connection was approved.

Logout removes the current session hash. Changing a password deletes sessions for the user, which signs out other devices as well.

11.4 Registration and patient profile flow (`app.py`)

`register_user()` validates the role and language, hashes the password, inserts a new user, creates a session, and—when the selected role is `caretaker`—creates a patient row owned by that account. It returns a public user object and the bearer token.

`current_user()` returns the authenticated user. It also repairs the special legacy case where a caretaker has no owned profile: it claims an unowned orphan patient row when one is available, or creates a new profile. This lets accounts from an earlier prototype continue into the current model.

`create_patient()` is role-limited so caregivers do not create a second person's patient record. Caregivers should request access to the patient's existing Caretaker account. `create_caregiver_request()` resolves the caretaker email to the patient's profile and creates/reopens a pending request. `update_caregiver_request()` checks the request and, on acceptance, creates the caregiver-to-patient link.

11.5 Medication plan and schedule flow (`app.py`)

`create_medicine()` validates the patient, optional prescription association, date, and each reminder time. If the user omitted reminder times, it calls `generate_schedule()` for a small set of familiar frequency patterns. The API returns a `schedule_needs_confirmation` indicator because a frequency-to-clock-time mapping is only a convenience and must be checked against the prescription.

`list_medicines()` reads the selected patient's medicines and normalizes stored reminder times before returning them to the browser.

`daily_schedule()` builds a schedule for a requested day. For each active medicine that has started and is still within its course, it creates one schedule item per reminder time. It checks `medication_logs` to decide whether an item is pending, taken, missed, or skipped, then sorts the result by clock time.

`record_dose()` validates the submitted date, time, and status. It looks up the corresponding medicine and any existing log for that date/time. If a log exists, it updates it; otherwise it inserts a new log. The SQL branches handle the old database's required `medicine_name` column as well as the newer table layout. Marking a dose missed also creates a notification for the patient and connected caregivers. The returned JSON object is the persisted log row.

The dose status endpoint is `POST /api/medicines/{medicine_id}/doses`. Example body:

```
{
  "scheduled_date": "2026-10-12",
  "scheduled_time": "08:00",
  "status": "taken",
  "note": null
}
```

The browser uses the selected schedule item's medicine ID, date, and time when the user presses **Taken** or **Missed**. Its shared `api()` helper adds the bearer token, sends JSON, parses the response, and raises a readable error for non-success responses.

11.6 Prescription image processing

The OCR route checks the MIME type, reads at most 10 MB plus one byte to detect oversized uploads, and rejects empty files. It calls `extract_text()` in `ocr.py`; that function reads `OCR_SPACE_API_KEY` from the environment and posts the image bytes to OCR.space with a timeout.

The returned text goes to `extract_medicines()` in `medicine_extractor.py`. This parser looks for common strength units, frequency expressions like `1-0-1`, durations in days, and basic meal timing phrases. It uses the optional local medicine dictionary for fuzzy name suggestions. Every candidate has review metadata and can be wrong: the API stores the prescription OCR record but does not create a medication plan at this stage.

In the browser, `uploadPrescription()` sends a multipart `FormData` upload. `renderOcrResults()` creates editable fields and review checkboxes. `saveOcrMedicines()` sends only rows selected by the user to the normal medicine creation route. That second step is the explicit human confirmation boundary.

11.7 Reminder worker and browser alerts

`run_reminder_worker()` loops until the app shuts down. Each iteration calls `process_due_items()` in a worker thread so blocking SQLite work does not run directly on the asynchronous event loop. Exceptions are logged and retried after the next polling delay.

`process_due_items()`:

1. Reads active medicines and parses the saved reminder times.
2. Skips medicine plans that have not started or have ended.
3. For each scheduled dose up to the current local time, checks whether a medication log already exists.
4. Within the grace window, creates a medicine reminder notification. After the grace window, it creates a missed-dose log and missed-dose notification.
5. Creates refill/course-ending notifications when the entered quantity or duration meets its estimate thresholds.
6. Dispatches due custom reminders and appointment reminders as durable notification rows.

`_notify()` targets the caretaker account and approved caregivers. Dedupe keys plus SQLite uniqueness prevent repeated worker cycles from creating the same notification over and over.

The frontend polls `/api/notifications` while the session is active and refreshes patient data to update the top-of-page due alert. Desktop notification APIs are browser-controlled: permission must be granted, and alerts require the browser page to be running. The current implementation is not an operating-system alarm service.

11.8 Frontend structure (`frontend/app.js`)

The frontend is a small single-page application without a build step:

- `state` holds the bearer token, current user, selected patient, current view, schedule, medicines, reports, notifications, and modal state.
- `api()` centralizes HTTP requests, adds authorization, handles expired sessions, parses JSON, and surfaces API errors.
- `renderAuth()` draws the login/register page. `shell()` draws the navigation frame and inserts the active view. `renderOverview()`, `renderMedications()`, `renderCare()`, and the other `render...()` functions return HTML strings for their page sections.
- A delegated click listener handles controls using `data-action` attributes. A delegated submit listener handles forms by ID. This avoids registering a new event handler each time HTML is re-rendered.
- `loadWorkspace()` requests the user, accessible patients, and selected-patient data after login. `loadPatientData()` makes the schedule, medicines, report, appointments, reminders, prescription, and profile requests.
- `refresh()` reloads data after a mutation such as saving a medicine or recording a dose.
- `startPolling()` refreshes notifications and schedule data periodically while signed in.
- `esc()` escapes interpolated content before it is inserted into HTML. User-provided text should continue to be escaped anywhere it enters an HTML template.
- `translations` maps a set of common English interface strings to Hindi and Marathi. `applyTranslations()` changes exact text nodes after rendering; it is not a complete translation system, so unmapped text remains English.

`frontend/index.html` supplies the app root and references CSS/JavaScript assets. `frontend/app.css` contains the responsive grid, sidebar, cards, forms, modals, toasts, and due-reminder banner.

11.9 Reports and estimates (`app.py`)

`adherence_report()` sums taken/missed/skipped logs per medicine for a date range. It computes adherence from taken and missed events and compares the entered course duration and quantity with elapsed time and reminder count to provide remaining-days and quantity estimates. These are simple estimates; a user should not use them as a clinical instruction or refill authorization.

11.10 Built-in chat and external lookup

`chat()` checks the message for a short list of emergency phrases. If one matches, it responds with a direction to seek local emergency help. Otherwise it returns fixed conservative language: it cannot diagnose, adjust prescriptions, or recommend doses. It is not a model-backed chatbot and does not use an API key.

`rxnorm_lookup()` calls the public RxNav REST endpoint with `requests`, applies a timeout, and returns matching RxNorm concept identifiers when available. A network/API failure returns an empty match list and an error indicator rather than crashing the rest of the app.

12. Example end-to-end API usage

Register and obtain a token

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Alex Example",
  "email": "alex@example.com",
  "role": "caretaker",
  "password": "use-a-long-local-password",
  "language": "en"
}
```

The response contains `access_token` and a `user` object. Caretaker registration creates the owned patient profile automatically.

Create a medicine plan

Send a bearer token to `POST /api/patients/{patient_id}/medicines`:

```
{
  "medicine_name": "Example medicine",
  "dose": "500 mg",
  "frequency": "1-0-1",
  "timing": "after food",
  "duration": 5,
  "start_date": "2026-10-12",
  "reminder_times": ["08:00", "20:00"],
  "quantity": 10
}
```

Load schedule and record a dose

`GET /api/patients/{patient_id}/schedule?on=2026-10-12` returns items with `medicine_id`, `medicine_name`, `scheduled_date`, `scheduled_time`, and `status`. Post the schedule item's ID/date/time to `/api/medicines/{medicine_id}/doses` to persist the outcome.

Call a protected route

```
GET /api/patients/1/medicines
Authorization: Bearer <access_token>
```

The numeric ID is an example only; use the ID returned for the signed-in user's accessible profile.

13. HTTP errors and debugging pointers

FastAPI validation failures commonly return `422`. Authentication failures return `401`; insufficient patient access returns `403`; missing records return `404`; conflicts such as duplicate registration can return `409`; external OCR failures are translated to a gateway/service error. A `500` indicates an unhandled server-side error: inspect the terminal running Uvicorn for the Python traceback. For database errors, check the actual schema with a SQLite browser or `PRAGMA table_info(table_name)` and confirm the process is using the expected `CAREMATE_DB` path. Do not paste patient data or API keys into public issue reports.

14. Dependency notes

`requirements.txt` declares:

- `fastapi`: route framework, validation integration, and docs.
- `uvicorn[standard]`: ASGI server and development reload support.
- `python-multipart`: multipart form parsing for image uploads.
- `requests`: outbound HTTP calls to OCR.space and RxNav.

SQLite, password hashing, token generation, date handling, and the reminder loop use Python standard-library modules. The browser UI uses built-in web APIs; there is no `npm install` or JavaScript bundling step.